

Vistas en Bases de Datos (MySQL)

Introducción

Las vistas en MySQL son una herramienta poderosa que permite a los usuarios interactuar con los datos de una manera más sencilla y segura. Una vista es una tabla virtual cuyo contenido se define mediante una consulta. A diferencia de una tabla, una vista no almacena datos físicamente, sino que genera los datos dinámicamente cada vez que se consulta.

¿Qué es una Vista?

Una vista es una consulta almacenada que se puede tratar como una tabla. Las vistas se utilizan para simplificar la complejidad de las consultas, proporcionar seguridad y presentar los datos de una manera específica.

Las vistas tienen varios usos, entre ellos:

Simplicidad: Simplificar consultas complejas al encapsularlas en una vista que se puede consultar fácilmente.

Seguridad: Restringir el acceso a datos sensibles proporcionando solo la información necesaria a través de la vista.

Mantenimiento: Facilitar el mantenimiento de consultas complejas al centralizarlas en una vista.

Consistencia: Asegurar que las consultas reutilizadas sean consistentes en toda la aplicación.

Características de las Vistas

1. **Es una Tabla Virtual:** Una vista no almacena datos físicamente, sino que muestra los datos de las tablas subyacentes.
2. **Actualización Dinámica:** Los datos en una vista siempre están actualizados, ya que se generan en tiempo real a partir de las tablas base.
3. **Seguridad:** Las vistas pueden restringir el acceso a datos sensibles al mostrar solo las columnas y filas necesarias.
4. **Simplificación:** Facilitan la escritura de consultas complejas al encapsular la lógica en una sola vista.
5. **Compatibilidad:** Proporciona una interfaz consistente incluso si cambia el esquema de las tablas subyacentes.
6. **Rendimiento:** Aunque MySQL no soporta vistas indexadas, las vistas pueden mejorar la organización y legibilidad de las consultas. También es importante tener en cuenta alternativas como las tablas temporales y la optimización de consultas, en casos de vistas muy complejas.

Tipos de Vistas

1. **Vistas Simples:** Basadas en una sola tabla. Pueden ser "actualizables", es decir usar update, delete e insert a través de ellas, pero solo si están basadas en 1 sola tabla y no usan: join/unión/disctint/subquersys/funciones de agregado.
2. **Vistas Complejas:** Basadas en múltiples tablas y pueden incluir funciones agregadas. No son "actualizables" (complejidad para asegurar consistencia e integridad).
3. **Vistas Materializadas:** MySQL no soporta vistas materializadas/persistentes nativamente, pero se pueden simular mediante tablas temporales y triggers.

Limitaciones de las Vistas

1. *No Almacenan Datos:* Las vistas no almacenan datos físicamente, lo que puede afectar el rendimiento en consultas muy complejas.
2. *Actualización:* No todas las vistas son "actualizables", especialmente las que incluyen funciones agregadas o combinaciones complejas.
3. *Rendimiento:* Consultas a vistas complejas pueden tener un rendimiento inferior en comparación con consultas directas a las tablas subyacentes, ya que la vista necesita ser recalculada cada vez que se accede. Es en estos (y otros casos), el **optimizador de consultas** puede que tenga alguna dificultad al momento de realizar su trabajo debido a que existen factores subyacentes a la vista, que afectan el rendimiento. Por ejemplo: complejidad de la consulta, índices, funciones agregadas, group by y tamaño de las tablas involucradas.
4. *Restricciones:* Algunas vistas no pueden contener ciertas cláusulas, como ORDER BY, a menos que se utilicen con LIMIT para garantizar el orden de los resultados.
5. *Seguridad y Permisos:* Las vistas pueden restringir el acceso a ciertas columnas de las tablas subyacentes, pero no pueden ocultar completamente los datos subyacentes si un usuario tiene permisos suficientes para acceder a las tablas directamente.
6. *Índices:* Las vistas no pueden tener índices propios (ya que no almacenan datos físicamente), lo que puede limitar el rendimiento en consultas que se beneficiarían del uso de índices.
7. *Dependencia de las Tablas Subyacentes:* Los cambios en las tablas subyacentes, como la modificación de la estructura o eliminación de columnas, pueden invalidar las vistas que dependen de esas tablas.
8. *Limitaciones en Consultas de Unión:* Vistas que usan UNION o UNION ALL pueden tener limitaciones en su capacidad para actualizar datos a través suyo.
9. *Limitaciones para depurar consultas c/vistas complejas:* Esto se debe a que la lógica de las vistas esta oculta en su definición y por ende esto dificulta el seguimiento de una consulta que utilice una vista.

Crear Vistas en MySQL

1- create view comprasXcliente as

```
select
  c.CLI_ID,
  c.APELLIDO,
  c.NOMBRES,
  v.FECHA_COMPRA
from
  clientes c,
  ventas v
where
  c.CLI_ID = v.CLI_ID
```

2- CREATE

[OR REPLACE]

VIEW nombre_vista [(columna1, columna2, ...)] //por defecto, los mismos
nombres que las columnas de las tablas

AS

sentencia_select

[WITH [CASCADED | LOCAL] CHECK OPTION];

```
CREATE OR REPLACE VIEW clientes_activos (id_cliente, nombre_completo, email) AS
SELECT cli_id, CONCAT(apellido, ' ', nombres), correo_electronico
FROM clientes
WHERE estado = 'activo';
```

With CHECK OPTION: se utiliza al crear o modificar una vista para controlar cómo se manejan las inserciones y actualizaciones de datos a través de esa vista. Su objetivo principal es asegurar que cualquier cambio realizado a través de la vista cumpla con las condiciones definidas en la consulta SELECT que define la vista.

LOCAL:

- Cuando se utiliza LOCAL, las comprobaciones de la cláusula WHERE (o cualquier otra condición de filtrado) se aplican solo a la vista específica en la que se define CHECK OPTION.
- Si la vista se basa en otra vista (una vista anidada), las comprobaciones de otras vistas anidadas no se tienen en cuenta.

CASCADED (dependiendo del dbms):

- Cuando se utiliza CASCADE, las comprobaciones se aplican no solo a la vista actual, sino también a todas las vistas subyacentes (vistas anidadas) que tengan CHECK OPTION definido.
- Esto asegura que cualquier cambio realizado a través de la vista cumpla con las condiciones de todas las vistas en la cadena de vistas anidadas.

Vamos con ejemplos...

```
CREATE OR REPLACE VIEW clientes_por_ciudad AS
SELECT CONCAT(nombre, ' ', apellido) AS nombre_completo, email
FROM Clientes
WHERE ciudad = 'Madrid'
WITH CHECK OPTION;
```

Aquí definimos una vista que establece como condición en el **where**, que la ciudad sea Madrid. La cláusula WITH CHECK OPTION (por defecto: LOCAL) asegura que cualquier inserción o actualización en la vista cumpla con la condición definida en el WHERE de la vista.

¿Qué sucede en este caso?

```
INSERT INTO clientes_por_ciudad (nombre_completo, email) VALUES ('Pedro Sánchez', 'pedro.sanchez@email.com');
```

La instrucción falla, porque al no insertar la ciudad, no es posible verificar que la misma sea Madrid; por lo tanto, la inserción no cumple con la condición de la vista (establecida en el where).

Por otra parte, si la vista fuese la siguiente:

```
CREATE OR REPLACE VIEW clientes_por_ciudad AS
SELECT CONCAT(nombre, ' ', apellido) AS nombre_completo, email, ciudad
FROM Clientes
WHERE ciudad = 'Madrid'
WITH CHECK OPTION;
```

Podríamos ejecutar...

```
INSERT INTO clientes_por_ciudad (nombre_completo, email, ciudad)
VALUES ('Pedro Sánchez', 'pedro.sanchez@email.com', 'Madrid');
```

En este caso, la instrucción NO fallara, ya que podemos establecer la ciudad y la misma, al ser Madrid, no infringe la condición del **where** de la vista.

¿Qué sucede en este **otro** caso?

```
CREATE OR REPLACE VIEW clientes_madrid AS
SELECT nombre, apellido, ciudad
FROM Clientes
WHERE ciudad = 'Madrid'
WITH CHECK OPTION CASCADE;
```

```
CREATE OR REPLACE VIEW clientes_primera_letra_a AS
SELECT nombre, apellido, ciudad
FROM clientes_madrid
WHERE nombre LIKE 'A%'
WITH CHECK OPTION CASCADE;
```

En este caso Con CASCADE, cualquier operación sobre la vista derivada (clientes_primera_letra_a) no solo verifica las condiciones de esa vista, sino también las condiciones de la vista base (clientes_madrid).

Entonces la siguiente instrucción fallara:

```
INSERT INTO clientes_primera_letra_a (nombre, apellido, ciudad)
VALUES ('Ana', 'Lopez', 'Barcelona');
```

Si bien el nombre comienza con 'A' (condición de la vista derivada); no cumple con que la ciudad sea 'Madrid'. No así con el siguiente ejemplo, en el cual se cumple con ambas condiciones y por ende la instrucción NO fallara.

```
INSERT INTO clientes_primera_letra_a (nombre, apellido, ciudad)
VALUES ('Ana', 'Lopez', 'Madrid');
```

Nota: CASCADE estará o no disponible dependiendo del DBMS

Consultar la vista (como cualquier tabla)

```
select
    *
from
    comprascliente
```

Ejemplos

Vista "Simple"

```
1- CREATE VIEW ResumenDeVentas AS
SELECT
    IDOrdenDeVenta AS SalesOrderID,
    FechaDeOrden AS OrderDate,
    TotalDebido AS TotalDue
FROM
    EncabezadoDeOrdenDeVenta;
```

```
2- CREATE TABLE empleados (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50),  
    departamento VARCHAR(50),  
    salario DECIMAL(10, 2)  
);
```

```
CREATE VIEW vista_empleados AS  
SELECT nombre, departamento  
FROM empleados;
```

```
-- Actualizar el departamento de un empleado  
UPDATE vista_empleados  
SET departamento = 'Recursos Humanos'  
WHERE nombre = 'Juan';
```

Vista "Compleja"

```
1- CREATE VIEW VentasDeProductos AS  
SELECT  
    p.Nombre,  
    SUM(s.CantidadOrdenada) AS TotalVendido  
FROM  
    Producto AS p  
JOIN  
    DetalleDeOrdenDeVenta AS s ON p.IDProducto = s.IDProducto  
GROUP BY  
    p.Nombre;
```

```
2- CREATE TABLE productos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50),  
    precio DECIMAL(10, 2)  
);
```

```
CREATE TABLE ventas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    producto_id INT,  
    cantidad INT,  
    FOREIGN KEY (producto_id) REFERENCES productos(id)  
);
```

```
CREATE VIEW vista_productos_ventas AS  
SELECT p.nombre, p.precio, SUM(v.cantidad) AS total_ventas  
FROM productos p  
JOIN ventas v ON p.id = v.producto_id  
GROUP BY p.nombre, p.precio;
```

```
-- Intentar actualizar a través de la vista generaria un error  
UPDATE vista_productos_ventas  
SET precio = 25.00  
WHERE nombre = 'Producto A';
```

Modificar una vista

- 1- DROP (borrarla)
- 2- CREATE (crearla nuevamente)

Eliminar una vista

DROP VIEW vista_empleados;